Processes

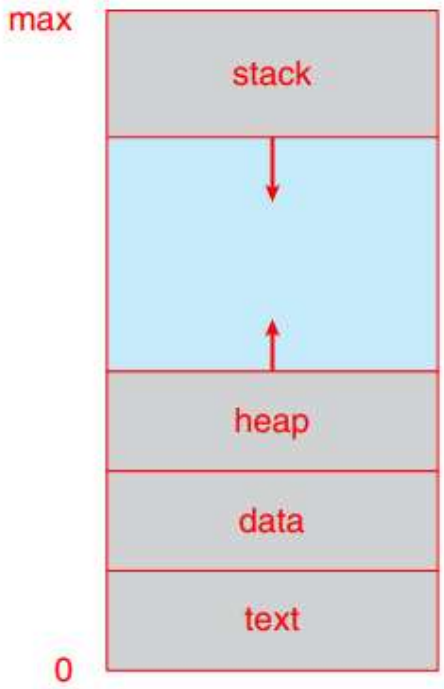
Process

What is a Process?

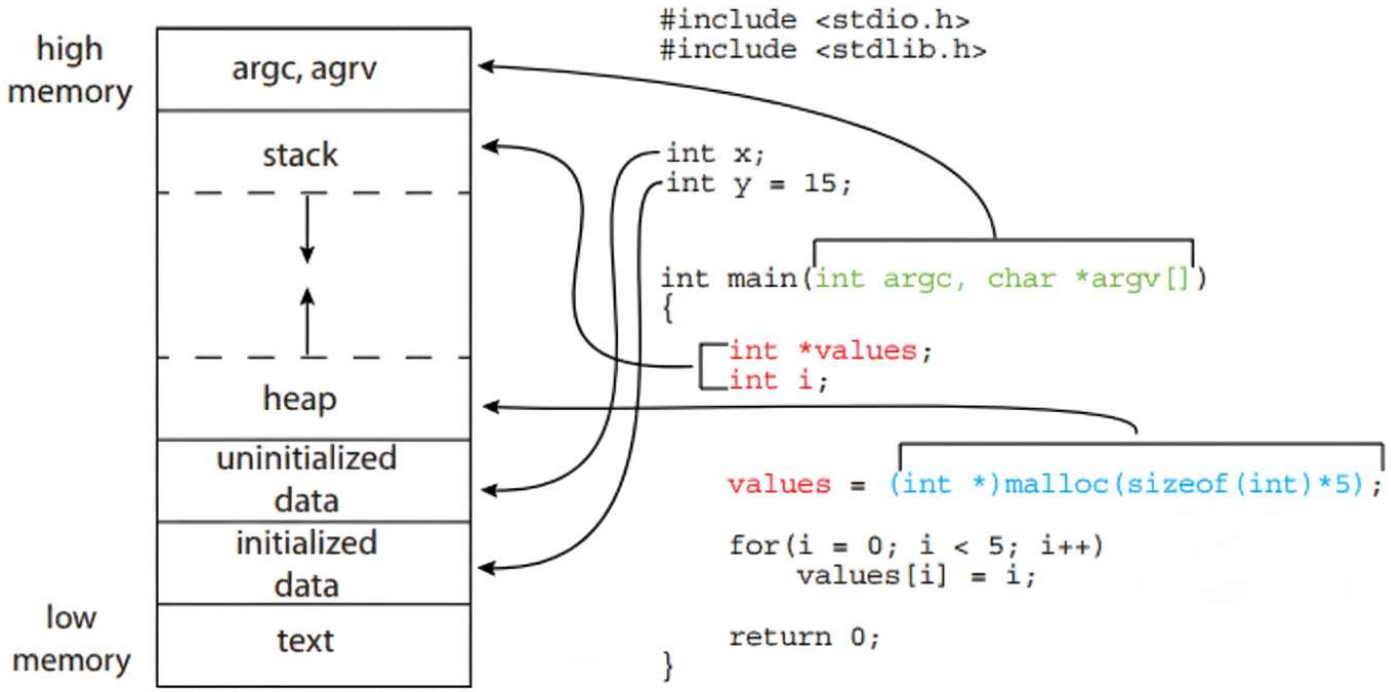
- A process is a program in execution
  - A program is a passive entity (like a file on a disk), while a process is an active one
- A single program, such as a web browser, can generate multiple processes to handle different tasks like separate tabs, each with its own state and resources

Process Components

- A process is composed of several key sections:
  - The **text section** contains the program code
  - The **data section** holds global variables
  - The **heap** is used for dynamically allocated memory
  - The **stack** is a temporary data storage area for function parameters, return addresses, and local variables

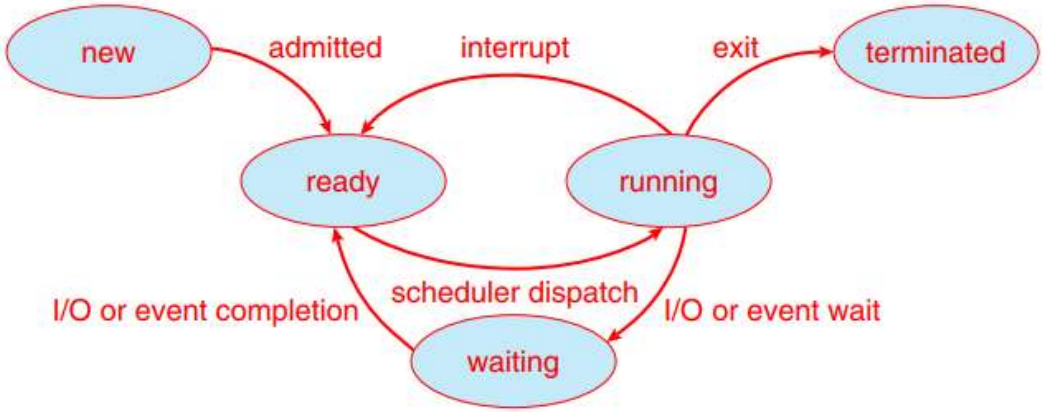


Memory layout of a C program



Process State

- As a process executes, its state changes, which is defined by its current activity. The states are:
  - New: The process is being created.
  - Running: Instructions are being executed by the CPU.
  - Waiting: The process is waiting for some event to occur (e.g., I/O completion or a signal).
  - Ready: The process is waiting to be assigned to a processor.
  - Terminated: The process has finished execution.



The Process Control Block

- The **Process Control Block (PCB)**, also known as a task control block, is a data structure in the operating system kernel that contains all the information needed to manage a specific process
  - It is the central repository for a process's state and is essential for the OS to perform a **context switch**.
- A PCB holds information about a process, including:
  - **Process State:** The current state of the process (e.g., New, Running, Waiting, Ready, Terminated)
  - **Program Counter (PC):** The address of the next instruction to be executed
  - **CPU Registers:** The values of all CPU registers, which must be saved and restored during a context switch
  - **CPU-Scheduling Information:** Information like the process priority, pointers to scheduling queues, and other scheduling parameters
  - **Memory-Management Information:** Pointers to the page tables or segment tables for the process
  - **Accounting Information:** CPU time used, time limits, and job numbers
  - **I/O Status Information:** A list of I/O devices allocated to the process and a list of open files



Threads

- A **thread** (or lightweight process) is a basic unit of CPU utilization
  - It is a single sequential flow of control within a process
  - A single-threaded process has only one flow of control, while a multi-threaded process has multiple
  - Threads within the same process share resources
- **Thread Components**
  - A thread consists of a **thread ID**, a **program counter**, a register set, and a **stack**
  - It does **not** have its own text, data, or heap sections

Benefits of Multithreading:

- **Responsiveness:** A multithreaded application can remain responsive even if part of it is blocked or performing a lengthy operation.
- **Resource Sharing:** Threads share the memory and resources of their parent process, making it more efficient than creating new processes.
- **Economy:** Creating and switching between threads is faster and less resource-intensive than creating and switching between processes.
- **Scalability:** Threads can run in parallel on multicore processors, taking full advantage of the hardware.

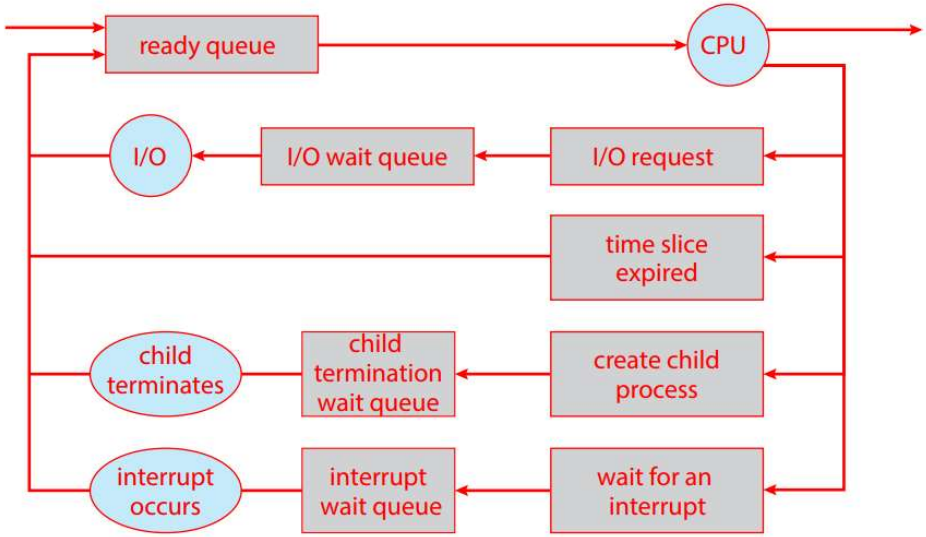
Multithreading Models

- Different models exist for mapping user-level threads to kernel threads:
  - **Many-to-One:** Many user threads map to a single kernel thread.
  - **One-to-One:** Each user thread maps to a separate kernel thread.
  - **Many-to-Many:** A flexible model where many user threads are mapped to a smaller or equal number of kernel threads.

Process Scheduling

Process Scheduling

- **The Goal of Scheduling:** The primary goal of a process scheduler is to select an available process for execution on the CPU. The OS uses various queues to manage processes:
  - The **job queue** holds all processes in the system
  - The **ready queue** holds all processes that are in main memory and are waiting for the CPU
  - **Device queues** hold processes that are waiting for an I/O device



Schedulers

- **Long-Term Scheduler (Job Scheduler):** This scheduler selects processes from storage and loads them into the ready queue. This controls the **degree of multiprogramming** (the number of processes in memory). It is invoked infrequently and is responsible for selecting a good mix of I/O-bound and CPU-bound processes.
- **Short-Term Scheduler (CPU Scheduler):** This scheduler selects from the processes in the ready queue and allocates the CPU to one of them. It is invoked very frequently and must be fast.
- **Medium-Term Scheduler:** Some operating systems also use a medium-term scheduler to remove processes from memory (swapping) and reintroduce them later.

Context Switch

- **Definition:** A **context switch** is the process of saving the state of the current running process and restoring the state of the next process to be run.
- **The Process:** When a context switch occurs, the OS saves the PCB of the old process and loads the PCB of the new one. This includes saving and restoring the program counter, CPU registers, and other process state information.
- **Overhead:** This is a pure overhead operation, as the system does no useful work during this time. The speed of a context switch depends on hardware support, such as the number of CPU registers.

Operations on Process

Process Creation

- **Parent-Child Relationship:** A process can create a child process using a system call, such as `fork()`
  - The creating process is the **parent process**, and the new process is the **child process**
- **Resource Sharing:** The parent and child processes can share resources in several ways:
  - The parent and child **share all resources** of the parent
  - The child process shares **a subset of the parent's resources**
  - The parent and child **share no resources** at all
- **Execution Options:** The parent and child processes can execute in one of two ways:
  - **Concurrent Execution:** The parent continues to execute concurrently with its children
  - **Parent Waits:** The parent waits until one or all of its children have terminated before continuing
- **Address Space:** The child process's address space can be a duplicate of the parent's or a new program can be loaded into the child's address space
  - The `fork()` and `exec()` system calls in UNIX-like systems are a prime example of this model

Process Termination

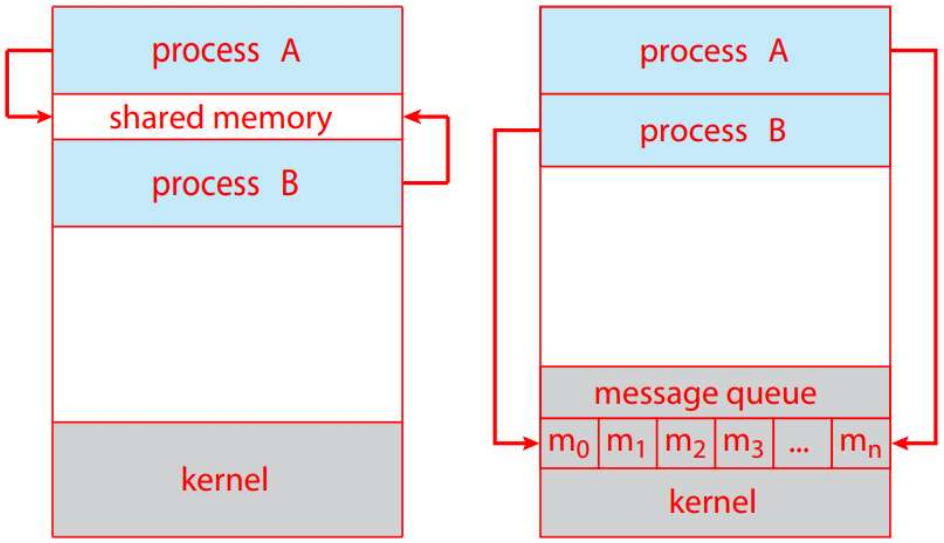
- **Normal Termination:** A process terminates when it finishes executing its last statement and uses the `exit()` system call to request termination
  - This returns a status value from the process to its parent
- **Parent-Initiated Termination:** A parent process can terminate one of its children for several reasons:
  - The child has exceeded its resource limits
  - The task assigned to the child is no longer needed
  - The parent is terminating, and the OS does not allow a child to continue if its parent terminates
- **Cascading Termination:** When a parent process terminates, some operating systems automatically terminate all its child processes as well
- **Zombie and Orphan Processes:**
  - A **zombie process** is a child that has terminated but whose parent has not yet called `wait()` to collect its exit status
  - An **orphan process** is a child whose parent has terminated without calling `wait()`
    - Orphans are often adopted by the init process, which collects their status and prevents them from becoming zombies

Interprocess Communication

- **Cooperating vs. Independent Processes:**
  - Independent processes don't share data, while cooperating processes do
  - **Interprocess Communication (IPC)** provides a mechanism for cooperating processes to exchange data
- **Benefits of IPC:**
  - **Information sharing:** Multiple applications can work on the same data
  - **Computation speedup:** A task can be broken into subtasks and run in parallel on multiple cores
  - **Modularity:** A system can be designed with different functions running in separate processes

Models of Communication in IPC

- **Shared Memory**
  - Processes establish a shared memory region
  - The OS is only involved in setting up this region; once established, communication is extremely fast, as it's done directly in memory without kernel intervention
- **Message Passing**
  - Processes communicate by exchanging messages
  - This is useful for exchanging smaller amounts of data and is the primary model for communication in distributed systems
  - The OS provides system calls like `send()` and `receive()`



Shared Memory Message Passing

Communication in Client-Server Systems

- **Sockets:**
  - Sockets are a standard way to implement IPC over a network
  - A socket is identified by a combination of an IP address and a port number
  - Sockets can be either **connection-oriented (TCP)** or **connectionless (UDP)**
- **Remote Procedure Calls (RPC)**
  - An **RPC** is a mechanism that allows a process to call a procedure (or function) on a remote machine as if it were a local function call
  - The RPC system handles all the details of network communication, packaging the call and its parameters into a message and sending it to the server
  - The client is typically blocked until the result is returned

Examples of IPC Systems

- **Pipes:** Pipes are a simple form of IPC that allows processes to communicate
  - **Ordinary Pipes:** These are **one-way** (unidirectional) and can only be used between a parent and a child process
  - **Named Pipes: Bidirectional** and can be used between unrelated processes on the same machine
- **POSIX Shared Memory**
  - A standardized set of APIs for creating and managing shared memory segments
  - The process that creates the segment and the processes that attach to it can use standard memory operations to read and write to the shared data
- **Mach**
  - An early operating system that was designed around the concept of message passing
  - Mach's microkernel architecture relies on messages for all inter-process communication, even for communication between the kernel and its services
- **Windows IPC**
  - Windows offers a rich set of IPC mechanisms, including message passing (for GUIs), shared memory, and named pipes

Processes